

Annexe C - Scenarios de demonstration

Ces scenarios sont prepares pour la demonstration pratique du site pendant la soutenance.

No	Scenario	Etapes	Resultat attendu
1	Accueil	Ouvrir http://127.0.0.1:5000	La page d'accueil du site s'affiche.
2	Connexion etudiant	student@elearn.com / student123	Le tableau de bord etudiant est accessible.
3	Cours et lecons	Ouvrir Courses puis un cours	Le catalogue et les lecons sont consultables.
4	Progression	Naviguer dans les lecons	La progression est sauvegardee automatiquement.
5	Quiz	Passer le quiz du cours	Score calcule et sauvegarde.
6	Examen final	Generer et soumettre 20 questions	Questions variees, correction et resultat sauvegarde.
7	Certificat	Score $\geq 70\%$ avec progression complete	Certificat disponible en telechargement.
8	Administration	admin@elearn.com / admin123 puis /admin	Gestion des cours, lecons et utilisateurs.
9	Securite roles	Ouvrir /admin avec un etudiant	Acces refuse / redirection.

Annexe D - Extraits de app.py

Ces extraits proviennent du fichier principal `app.py` et montrent les parties essentielles du fonctionnement du site.

Extrait D.1 - Configuration Flask et base de donnees

```
app = Flask(__name__)
app.secret_key = "elearn_demo_secret_key_not_for_prod"
EXAM_CACHE = {}

DATABASE = os.path.join(os.path.dirname(__file__), "elearn.db")

GEMINI_API_KEY = "AIzaSyDxEkl4iuoqg8HlfeJL_0B25-rh0JY_EIU"

# ████████████████████████████████████████████████████████
# COURSES & LESSONS DATA
# ████████████████████████████████████████████████████████
STATIC_COURSES = [
    {"id": "course1", "title": "Web Development Fundamentals", "desc": "Start your web journey with HTML, CSS, and JavaScript. Build real projects."},
    {
        "lessons": [
            {"id": "l1", "title": "Introduction to HTML", "desc": "HTML (HyperText Markup Language) is the foundation of every web page. In this lesson, we'll explore the basic elements of HTML, including tags, attributes, and how to structure content like headings, paragraphs, and links."},
            {"id": "l2", "title": "CSS Styling Basics", "desc": "CSS (Cascading Style Sheets) controls the visual presentation of your HTML documents. We'll cover selectors, properties, values, and how to organize styles using external sheets."},
            {"id": "l3", "title": "JavaScript Basics", "desc": "JavaScript adds interactivity and logic to your web pages.\\n\\n• Variables: var, let, const"}],
    },
    {"id": "course2", "title": "Python for Data Science", "desc": "Learn Python programming and apply it to real-world data analysis and visualization tasks."},
    {
        "lessons": [
            {"id": "l1", "title": "Python Fundamentals", "desc": "Python is a high-level, interpreted programming language known for its clear syntax and readability. This lesson covers basic syntax, variables, loops, and functions."}
        ]
    }
]
```

Extrait D.2 - Creation des tables SQLite

```
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    email TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    name TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'student'
);
CREATE TABLE IF NOT EXISTS notes (
    id TEXT PRIMARY KEY,
    user_id INTEGER NOT NULL,
    title TEXT,
    content TEXT,
    color TEXT DEFAULT 'cyan',
    date TEXT
);
CREATE TABLE IF NOT EXISTS courses (
    id TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    description TEXT,
    emoji TEXT DEFAULT 'book',
    color TEXT DEFAULT '#1a2a4a',
    level TEXT DEFAULT 'beginner',
    hours INTEGER DEFAULT 0,
    category TEXT DEFAULT 'general',
    created_by INTEGER,
    created_at TEXT
);
CREATE TABLE IF NOT EXISTS lessons (
    id TEXT PRIMARY KEY,
    course_id TEXT NOT NULL,
    title TEXT NOT NULL,
    content TEXT,
    lesson_order INTEGER DEFAULT 0
);
CREATE TABLE IF NOT EXISTS progress (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    course_id TEXT NOT NULL,
    lesson_id TEXT NOT NULL,
    visited_at TEXT NOT NULL,
    UNIQUE(user_id, lesson_id)
);
CREATE TABLE IF NOT EXISTS quiz_results (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    course_id TEXT NOT NULL,
    score INTEGER NOT NULL,
    total INTEGER NOT NULL,
    percentage REAL NOT NULL,
    taken_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS completions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    course_id TEXT NOT NULL,
    quiz_score REAL NOT NULL,
    certificate_id TEXT UNIQUE NOT NULL,
    completed_at TEXT NOT NULL,
    UNIQUE(user_id, course_id)
);
CREATE TABLE IF NOT EXISTS exam_results (
```

```
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER,
        course_id TEXT,
        score INTEGER,
        total INTEGER,
        percentage REAL,
        created_at TEXT
    );
CREATE TABLE IF NOT EXISTS exam_completions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    course_id TEXT,
    percentage REAL,
    certificate_id TEXT UNIQUE,
    created_at TEXT,
    UNIQUE(user_id, course_id)
);
""")
```

Extrait D.3 - Controle d'accès

```
def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if "user_id" not in session:
            flash("Please sign in to continue.", "info")
            return redirect(url_for("login"))
        return f(*args, **kwargs)
    return decorated

def admin_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if "user_id" not in session:
            return redirect(url_for("login"))
        if session.get("role") != "admin":
            flash("Admin access required.", "danger")
            return redirect(url_for("dashboard"))
        return f(*args, **kwargs)
    return decorated
```

[illegible][illegible]